



SpendWiser

COMPLETE TECHNICAL DOCUMENTATION

MERN STACK

REACT 19 + TYPESCRIPT

GOOGLE GEMINI AI

MONGODB + EXPRESS

A full-stack personal finance management web application featuring AI-powered spending insights, SMS-based transaction parsing, investment recommendations, and multi-wallet tracking — all backed by MongoDB and served with an Express REST API.

 June 2026

 Software Design Engineer Documentation

 Version 2.5.0



Table of Contents

1. Project Overview

1.1 What is SpendWiser?

1.2 Key Features

1.3 Technology Stack

2. System Architecture

2.1 High-Level Architecture

2.2 Project Folder Structure

2.3 Data Flow

3. Backend – Express API

3.1 Entry Point – server.js

3.2 Database Connection – db.js

3.3 Database Models

3.4 API Routes

3.5 Controllers

3.6 Middleware

3.7 Services

3.8 Utilities

3.9 Validators

4. Frontend – React Application

4.1 Main App Component (App.tsx)

4.2 TypeScript Types

4.3 Client Services

5. Pages and Components

5.1 Login / Register Page

5.2 Sidebar Navigation

5.3 Dashboard (Neural Overview)

5.4 Vault Repository (Wallets)

5.5 Ledger Feed (Transactions)

5.6 Gemini Core (AI Insights)

5.7 AI Advisor (Suggestions)

5.8 Investment Plans

5.9 User Profile

5.10 Settings

5.11 Transaction Modal

5.12 SMS Parser Modal

6. API Reference

6.1 Authentication Endpoints

6.2 Transaction Endpoints

6.3 Bank Details Endpoints

6.4 SMS Webhook

6.5 Investment Rates

7. Security

8. Data Storage & Backup

9. Deployment

10. Setup & Installation Guide

11. Environment Variables

12. Troubleshooting



SECTION 1

Project Overview

Understanding what SpendWiser is, what it does, and the core technologies used.

1.1 What is SpendWiser?

SpendWiser is a full-stack, AI-powered personal finance management application built using the **MERN stack** (MongoDB, Express, React, Node.js). It helps individuals *track their income and expenses*, understand their spending patterns through *AI-generated insights*, connect their bank accounts to receive automatic transactions via SMS parsing, and explore curated *investment plans* tailored to their financial profile.

The application features a **futuristic dark-mode UI** with glassmorphism design, animated components, and a "neural/cyber" aesthetic — making finance feel modern and engaging.

💡 **WHO IS THIS FOR?**

SpendWiser is built for students, working professionals, or anyone who wants a smart, beautiful, and AI-assisted way to understand and control their money — primarily targeting the Indian market (₹ INR currency, Indian investment schemes like PPF, NPS, LIC).

1.2 Key Features

FEATURE CATEGORY	FEATURE	DESCRIPTION
💰 Transactions	Manual Entry	Add income/expense with amount, category, date, description, wallet selection
	SMS Parsing	Paste any bank SMS — AI (Gemini) or regex engine auto-extracts amount, type, merchant

FEATURE CATEGORY	FEATURE	DESCRIPTION
	Receipt Scan	Upload a receipt image; Gemini AI reads merchant name, amount, date, category
	Excel Download	Download all transactions as .xlsx spreadsheet
	CSV Export	Export filtered transactions to .csv from the browser
 Banking	Bank Account Storage	Store up to 10 bank accounts (name, IFSC, account type, mobile). Secured in MongoDB.
	Account Masking	Account numbers display only last 4 digits (e.g., ****4321)
	SMS Webhook	Receive real bank SMS alerts automatically via a webhook endpoint
 AI Features	Spending Insights	Google Gemini analyzes your transactions and gives a 3-sentence financial summary
	AI Suggestions	Gemini returns 4–6 structured cards: saving tips, investment ideas, warnings
	Bank Message Parser	Gemini reads a raw bank SMS and extracts structured transaction data
 Analytics	Dashboard Charts	Bar chart showing spending by category; liquid gauge budget meters
	Budget Tracking	Auto-calculates how much you've spent in each budget category
	Live Investment Rates	Background scraper fetches real PPF interest rate from ClearTax.in every 2 months
 Settings	Toggle Preferences	Push notifications, email alerts, sound effects, auto-sync, 2FA, data retention
	Data Management	Export full JSON backup or permanently delete all data

1.3 Technology Stack

FRONTEND FRAMEWORK

React 19

with TypeScript 5.8, Vite 6.2 build tool

STYLING

Tailwind CSS

Utility-first CSS, dark glassmorphism theme

CHARTS

Recharts 3.5

Bar charts, responsive containers, custom tooltips

ICONS

Lucide React 0.554

500+ SVG icons as React components

BACKEND RUNTIME

Node.js + Express 5

REST API, CORS, rate limiting, error handling

DATABASE

MongoDB + Mongoose 8

ODM for schema definition, queries, upsert operations

AUTHENTICATION

JWT (jsonwebtoken)

Bearer token auth, 7-day expiry, bcryptjs password hashing

AI ENGINE

Google Gemini API

gemini-2.5-flash and gemini-2.0-flash models

WEB SCRAPING

Axios + Cheerio

Fetches live PPF rates from ClearTax.in every 2 months

EMAIL

Nodemailer (SMTP)

Sends welcome email on new user registration

VALIDATION

Joi 17

Schema-based request body validation

SCHEDULING

node-cron 4

Cron job: scrape investment rates every 2 months

EXCEL

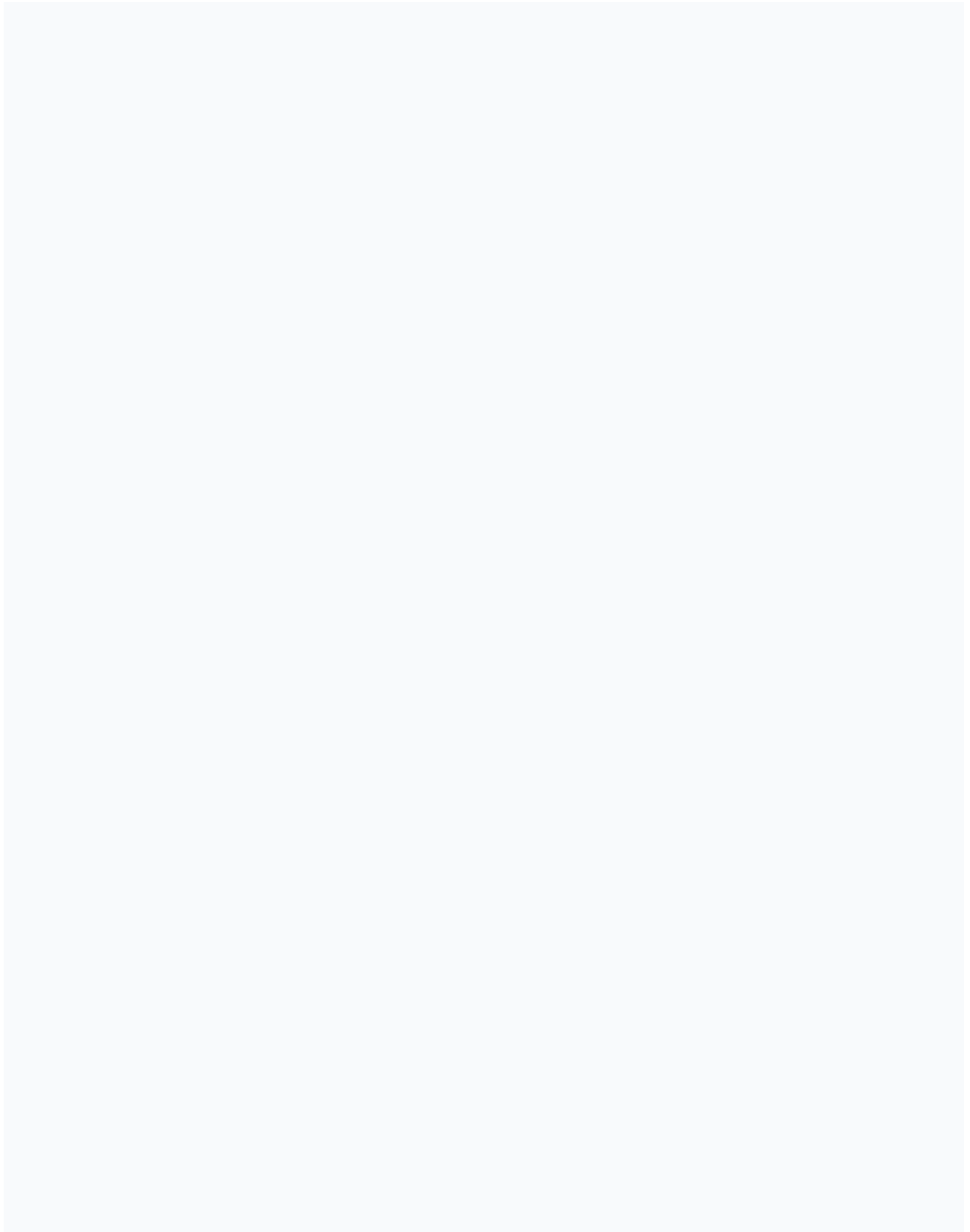
xlsx (SheetJS)

Read/write .xlsx as offline backup storage

DEPLOYMENT

Vercel

Frontend static + serverless backend via
vercel.json



SECTION 2

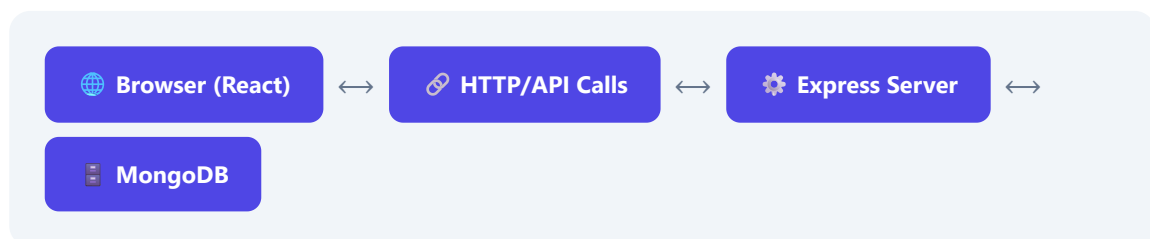
System Architecture

How all the pieces fit together — from browser to database.

2.1 High-Level Architecture

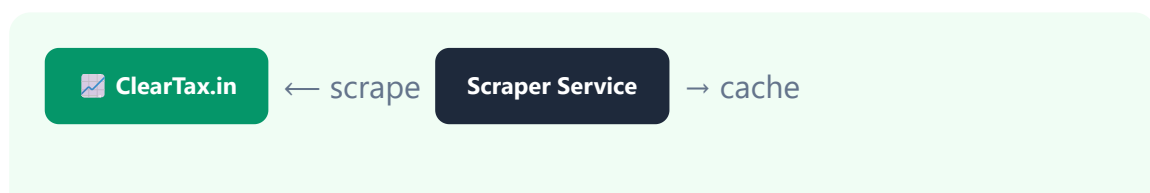
SpendWiser follows a **client-server architecture** with three main layers:

1. **Client (Browser)** — React 19 SPA running in the user's browser. Handles all UI rendering, local state, and makes API calls to the backend.
2. **Backend (Express Server)** — Node.js/Express REST API running on port 5000. Handles business logic, authentication, data persistence, and AI calls.
3. **Database (MongoDB)** — Stores Users, Transactions, BankDetails, and investment Rates documents.



✦ IMPORTANT NOTE ON AI CALLS

All AI calls (Gemini API) are made **directly from the browser** using the `VITE_GEMINI_API_KEY` environment variable. This means the AI features work completely client-side without passing data through the backend.



MongoDB / rates.json

→ serve

React App

2.2 Project Folder Structure

```
spendwiser/
├─ backend/ ← Node.js + Express API server
│  ├─ server.js ← Main entry point (Express app)
│  ├─ controllers/ ← Business logic per feature
│  │  ├─ authController.js ← register, login, getUser, updateUser
│  │  ├─ transactionController.js ← CRUD + download + clear
│  │  ├─ bankController.js ← save/get/delete bank details
│  │  └─ smsController.js ← receive SMS webhook
│  ├─ models/ ← Mongoose schemas (MongoDB collections)
│  │  ├─ User.js
│  │  ├─ Transaction.js
│  │  ├─ BankDetails.js
│  │  └─ Rates.js
│  ├─ routes/ ← Express Router definitions
│  │  ├─ authRoutes.js
│  │  ├─ transactionRoutes.js
│  │  ├─ bankRoutes.js
│  │  └─ smsRoutes.js
│  ├─ middleware/ ← Request processing layers
│  │  ├─ auth.js ← JWT token verification
│  │  ├─ rateLimiter.js ← Rate limiting (4 levels)
│  │  └─ validate.js ← Joi request validation
│  ├─ services/ ← External service integrations
│  │  ├─ emailService.js ← Nodemailer SMTP welcome email
│  │  ├─ scraperService.js ← Web scraping + caching investment rates
│  │  └─ excelService.js ← SheetJS read/write .xlsx backup
│  ├─ utils/ ← Reusable helper functions
│  │  ├─ db.js ← MongoDB connection manager
│  │  ├─ smsParser.js ← Regex-based bank SMS parser
│  │  ├─ excelHelper.js
│  │  └─ paths.js
│  └─ validators/
│     └─ schemas.js ← Joi validation schemas
├─ .env ← Secret environment variables
└─ data.xlsx ← Fallback Excel backup file
```

```

| └─ rates.json ← Cached investment rates
|
|─ client/ ← React 19 + TypeScript SPA
|  |─ index.html ← HTML shell
|  |─ index.tsx ← React DOM root render
|  |─ App.tsx ← Root component (state management + routing)
|  |─ types.ts ← TypeScript interface definitions
|  |─ index.css ← Global Tailwind + custom styles
|  |─ components/ ← All page-level React components
|  |  |─ LoginPage.tsx
|  |  |─ Sidebar.tsx
|  |  |─ Dashboard.tsx
|  |  |─ VaultsView.tsx
|  |  |─ LedgerView.tsx
|  |  |─ InsightsView.tsx
|  |  |─ AISuggestionsView.tsx
|  |  |─ InvestmentPlansView.tsx
|  |  |─ UserProfileView.tsx
|  |  |─ SettingsView.tsx
|  |  |─ TransactionModal.tsx
|  |  |─ MessageParserModal.tsx
|  |  |─ SMSIntegrationModal.tsx
|  |  └─ LiquidGauge.tsx
|  └─ services/
|     └─ authService.ts ← Login/register/logout API wrappers
|     └─ geminiService.ts ← All Google Gemini AI calls
|
|─ api/ ← Vercel serverless entry
|  └─ index.js ← Re-exports backend app for Vercel Functions
|
|─ vercel.json ← Vercel routing config
|─ package.json ← Root scripts (dev, build)
└─ README.md

```

2.3 Data Flow — Step by Step

When User Adds a Transaction

1. User clicks **"Initialize Log"** button in the header → `TransactionModal` opens
2. User fills in: Amount, Type (Income/Expense), Category, Description, Wallet, Date → clicks "Authorize Protocol"

3. React `addTransaction()` is called in `App.tsx`
4. UI is **optimistically updated** immediately (no waiting for server)
5. An HTTP `POST /api/transactions` request is sent with JWT Bearer token
6. Express backend receives the request → JWT middleware verifies token → Joi validates body
7. If MongoDB is connected → saves to `Transaction` collection; else falls back to Excel file
8. Wallet balance is updated locally in React state
9. Budget spending is auto-recalculated from the new transactions array

When User Clicks "Get AI Suggestions"

1. User navigates to "AI Advisor" tab → clicks "Get AI Suggestions"
2. React calls `getAISuggestions(transactions)` in `geminiService.ts`
3. The service calculates: total income, total expenses, savings rate %, category breakdown
4. This data is sent as a prompt to **Google Gemini 2.5 Flash** with a JSON response schema
5. Gemini returns: a summary + 4–6 suggestion cards (saving/investment/warning types)
6. React renders these as styled suggestion cards with priority badges and impact estimates

SECTION 3

Backend – Express API

Every file, every function, every configuration explained.

3.1 Entry Point — server.js

The file `backend/server.js` is the **heart of the backend**. It starts the Express application, connects everything together, and begins listening for requests.

What it does, line by line:

1. `require('dotenv').config()` — loads all variables from `.env` file so they are available as `process.env.X`
2. Imports Express, CORS, Mongoose, node-cron, path
3. Imports middleware: `apiLimiter` (rate limiter)
4. Imports services: `scraperService` (investment rates), `excelService` (Excel file path)
5. Imports routes for: transactions, auth, bank-details, SMS
6. Creates the Express app with `app.use(express.json())` — allows reading JSON request bodies
7. Configures **CORS**: only allows requests from `FRONTEND_URL` (default: localhost:3001), supports credentials, allows specific HTTP methods and headers
8. Applies rate limiter to all `/api/` routes — max 100 requests per 15 minutes
9. Calls `loadCachedRates()` immediately — fills investment rate cache from file/defaults
10. Schedules a cron job: `0 0 1 */2 *` = midnight on the 1st of every 2nd month → runs `performScraping()`
11. Runs `performScraping()` once immediately on server start
12. Registers a fast endpoint: `GET /api/investment-rates/scrape` → returns cached rates (no DB needed)
13. Connects to MongoDB asynchronously at startup; once connected, loads rates from DB
14. Adds a DB connection middleware for all `/api` routes — ensures DB is connected before handling requests (with graceful fallback)

15. Registers all route handlers at their URL prefixes
16. Registers a **global error handler** as the last middleware — catches any unhandled errors and returns 500
17. Starts listening on port 5000 (or `process.env.PORT`) *only if not in production* — on Vercel, the serverless function handles this
18. `module.exports = app` — exports the app for use in Vercel's serverless entry

```
// Key startup sequence
loadCachedRates();           // 1. Fill rate cache from file
cron.schedule('0 0 1 */2 *', performScraping); // 2. Schedule refresh
performScraping();           // 3. Run immediately
connectDB().then(loadRatesFromDB); // 4. Connect to Mongo + load DB rates
```

3.2 Database Connection — utils/db.js

This utility manages a **singleton MongoDB connection** using Mongoose. It prevents opening multiple connections if the server is already connected.

- `connectDB()` — checks `mongoose.connection.readyState`:
 - `1` = already connected → returns existing connection immediately
 - `2` = connecting → awaits the existing promise (no duplicate connections)
 - Otherwise → creates a new connection with `mongoose.connect()`
- For **localhost/127.0.0.1** URLs, forces IPv4 (`family: 4`) to avoid DNS resolution issues on Windows
- Stores the last connection error with `lastError` so controllers can report it to the client
- `getLastError()` — exported function that controllers use to give useful error messages

⚠️ FALLBACK MODE

If MongoDB is unreachable, the app does NOT crash. Instead, controllers detect `readyState !== 1` and fall back to reading/writing the `data.xlsx` Excel file. This means the app works even without a database!

3.3 Database Models

Mongoose models define the structure of documents stored in MongoDB collections.

User Model (`models/User.js`)

FIELD	TYPE	RULES	PURPOSE
<code>id</code>	String	unique, required	UUID-based custom ID (not MongoDB's default <code>_id</code>)
<code>email</code>	String	unique, required, lowercase	User login identifier
<code>password</code>	String	required	Bcrypt-hashed password (never stored plain)
<code>name</code>	String	optional	Display name
<code>createdAt</code>	Date	default: now	Account creation timestamp
<code>updatedAt</code>	Date	default: now	Last modification timestamp

Special behaviors:

- **Pre-save hook:** Before saving, if password was changed, it runs `bcrypt.genSalt(10)` + `bcrypt.hash()` automatically — so you never need to manually hash passwords
- `comparePassword(candidatePassword)` method — compares a plain text password with the stored hash using `bcrypt.compare()`

Transaction Model (`models/Transaction.js`)

FIELD	TYPE	DEFAULT	PURPOSE
<code>id</code>	String	<code>uuidv4()</code>	UUID auto-generated on creation
<code>amount</code>	Number	—	Transaction amount in specified currency
<code>type</code>	String	—	'income' or 'expense'
<code>category</code>	String	—	Food, Transport, Housing, Entertainment, Shopping, Health, etc.
<code>description</code>	String	—	Human-readable label (merchant name, note)
<code>date</code>	String	—	ISO date string (YYYY-MM-DD)

FIELD	TYPE	DEFAULT	PURPOSE
walletId	String	—	Links transaction to a wallet (1=checking, 2=cash, 3=savings)
currency	String	'INR'	Currency code
source	String	'manual'	'manual', 'sms', 'api', 'bank_sync'
smsFrom	String	—	Sender phone number (for SMS-sourced transactions)
createdAt	Date	now	Creation timestamp
updatedAt	Date	now	Last update timestamp

BankDetails Model (`models/BankDetails.js`)

FIELD	TYPE	RULES	PURPOSE
userId	String	required, unique	Links bank accounts to a specific user (one doc per user)
accounts	Array	default: []	List of bank account objects (up to 10 allowed by UI)
createdAt	Date	now	First save timestamp
updatedAt	Date	now	Last update timestamp

Each account in `accounts[]` contains: `bankName` , `accountNumber` , `ifscCode` , `accountType` , `mobileNumber` , `email` , `accountHolder` .

Rates Model (`models/Rates.js`)

Stores scraped investment interest rates as a key-value Map in MongoDB. Used to persist the scraped PPF/FD/NPS rates across server restarts.

3.4 API Routes

Routes connect HTTP endpoint paths to the correct controller functions, while applying appropriate middleware.

Auth Routes (routes/authRoutes.js)

HTTP METHOD	PATH	MIDDLEWARE CHAIN	CONTROLLER
POST	/api/auth/register	registerLimiter → validateRequest(userSchema)	authController.register
POST	/api/auth/login	loginLimiter → validateRequest(userSchema)	authController.login
GET	/api/auth/user/:userId	authMiddleware (JWT check)	authController.getUser
PUT	/api/auth/user/:userId	authMiddleware (JWT check)	authController.updateUser

Transaction Routes (routes/transactionRoutes.js)

HTTP METHOD	PATH	AUTH REQUIRED	CONTROLLER
GET	/api/transactions/	✔ JWT	getTransactions (sorted by date desc)
POST	/api/transactions/	✔ JWT + Joi validate	createTransaction
PUT	/api/transactions/:id	✔ JWT	updateTransaction
DELETE	/api/transactions/:id	✔ JWT	deleteTransaction
GET	/api/transactions/download	✗ None	downloadTransactions (returns .xlsx file)
DELETE	/api/transactions/data/clear	✗ None	clearAllTransactions (admin function)

Bank Routes (routes/bankRoutes.js)

HTTP METHOD	PATH	AUTH REQUIRED	CONTROLLER
GET	/api/bank-details/:userId	✔ JWT	getBankDetails

HTTP METHOD	PATH	AUTH REQUIRED	CONTROLLER
POST	/api/bank-details/	✓ JWT + Joi validate	updateBankDetails (upsert)
DELETE	/api/bank-details/:userId	✓ JWT	deleteBankDetails

SMS Routes (`routes/smsRoutes.js`)

HTTP METHOD	PATH	AUTH REQUIRED	CONTROLLER
POST	/api/sms/webhook	✗ None (webhook)	smsController.receiveWebhook

3.5 Controllers

Auth Controller (`controllers/authController.js`)

Handles all user account operations:

- **register(req, res)** — Creates a new user. Checks for duplicate email, creates User document (password auto-hashed via pre-save hook), sends welcome email asynchronously, returns user data (without password)
- **login(req, res)** — Finds user by email (lowercase), calls `user.comparePassword()` to verify, generates a JWT token with `userId` and `email` payload, 7-day expiry, returns `{ token, user }`
- **getUser(req, res)** — Fetches user by userId param (JWT protected)
- **updateUser(req, res)** — Updates name and/or password (JWT protected). Uses `findOneAndUpdate` with `{ new: true }` to return updated document

⚠ ALL CONTROLLERS CHECK MONGODB STATE

Every controller function starts with: `if (mongoose.connection.readyState === 1 || readyState === 2)`. If not connected, it returns a 503 error with the last connection error message. Transaction controller additionally falls back to Excel file operations.

Transaction Controller (`controllers/transactionController.js`)

- **getTransactions** — Finds all transactions, sorted by date descending. Falls back to reading Excel file if no DB
- **createTransaction** — Generates UUID, sets defaults (currency='INR'), saves to MongoDB or Excel fallback
- **updateTransaction** — Uses `findOneAndUpdate({ id })` with body spread + updated timestamp
- **deleteTransaction** — Uses `findOneAndDelete({ id })`
- **clearAllTransactions** — `Transaction.deleteMany({})` or recreates empty Excel sheet
- **downloadTransactions** — Calls `res.download(EXCEL_FILE, 'spendwiser-transactions.xlsx')`

Bank Controller (`controllers/bankController.js`)

- **getBankDetails(userId)** — `BankDetails.findOne({ userId })`
- **updateBankDetails** — Uses `findOneAndUpdate({ userId }, data, { upsert: true, new: true })` — creates OR updates in one query. On first bank save (no existing `bank_sync` transactions), inserts 4 sample transactions to populate the dashboard with demo data
- **deleteBankDetails(userId)** — `findOneAndDelete({ userId })`

SMS Controller (`controllers/smsController.js`)

- **receiveWebhook** — Accepts `{ message, from, timestamp }` in POST body. Calls `parseBankSMS(message)` from `smsParser.js` to extract transaction data. Creates a Transaction document with `source: 'sms'` and `smsFrom: from`. Saves to MongoDB or Excel fallback.

3.6 Middleware

JWT Auth Middleware (`middleware/auth.js`)

Verifies that the request has a valid JWT token:

1. Reads the `Authorization` header
2. Checks it starts with `"Bearer "`
3. Extracts the token (everything after "Bearer ")
4. Calls `jwt.verify(token, JWT_SECRET)`
5. If valid: adds decoded payload to `req.user` and calls `next()`
6. If invalid/expired: returns 401 with specific error message

Rate Limiter Middleware (`middleware/rateLimiter.js`)

Four different rate limiters for different security needs:

LIMITER	WINDOW	MAX REQUESTS	APPLIED TO
<code>apiLimiter</code>	15 minutes	100	All /api/ routes (global)
<code>loginLimiter</code>	15 minutes	5	/api/auth/login only
<code>registerLimiter</code>	1 hour	3	/api/auth/register only
<code>smsLimiter</code>	1 hour	50	/api/sms/webhook

Validation Middleware (`middleware/validate.js`)

A higher-order function: `validateRequest(schema)` returns an Express middleware that runs `schema.validate(req.body)` . If validation fails, returns 400 with Joi's error details. If valid, calls `next()` .

3.7 Services

Email Service (`services/emailService.js`)

- Creates a Nodemailer transporter using SMTP credentials from environment variables
- `sendWelcomeEmail(userEmail, userName)` — sends HTML welcome email after registration
- Gracefully skips if `SMTP_USER` or `SMTP_PASS` are not configured
- Email template includes the user's name and a welcome message

Scraper Service (`services/scraperService.js`)

- Maintains an in-memory `cachedRates` object with default values for PPF (7.1%), FD (6.5%), NPS (9%), LIC (5.5%), Gold (10.5%), Mutual Funds (14%)
- `loadCachedRates()` — reads `rates.json` file on disk and merges into cache
- `loadRatesFromDB()` — queries the Rates MongoDB document and merges into cache
- `performScraping()` — uses Axios to fetch ClearTax.in/ppf page, Cheerio to parse HTML, extracts PPF rate with regex. Saves updated rates to file and MongoDB
- `getCachedRates()` — returns current cached rates for the API endpoint

Excel Service (`services/excelService.js`)

- Uses SheetJS (xlsx library) to manage the `data.xlsx` file
- `initExcel()` — creates the file with headers if it doesn't exist
- `readTransactionsFromExcel()` — reads all rows and converts to Transaction objects
- `writeTransactionsToExcel(transactions)` — overwrites the sheet with the given array
- Acts as a **complete fallback database** when MongoDB is unavailable

3.8 Utilities

SMS Parser (`utils/smsParser.js`)

A pure function that parses raw bank SMS text using regular expressions — no AI needed:

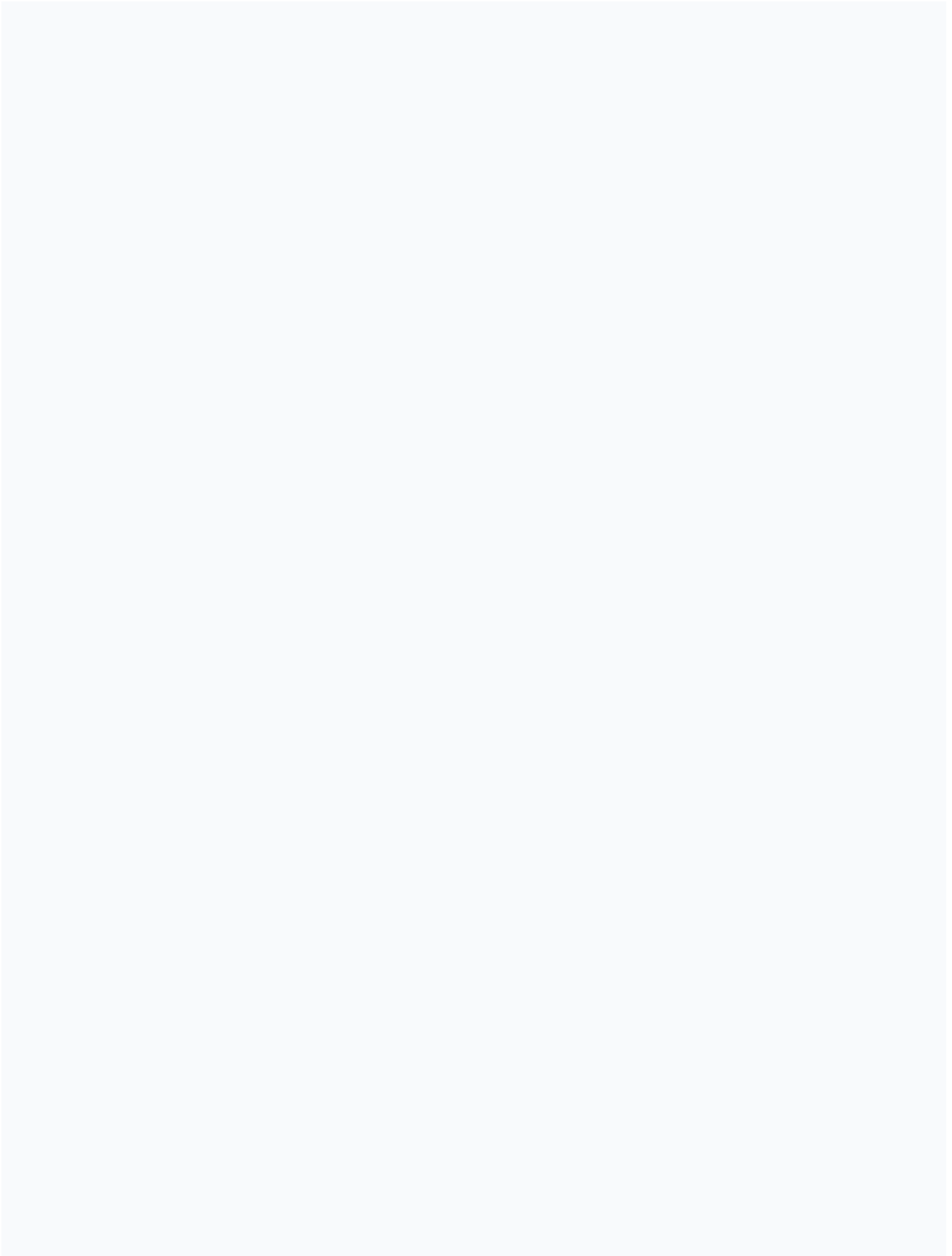
- Defines 4 regex patterns to match: "Paid Rs. X", "Debit/Credit of Rs. X from/to BANK", "Amount Rs. X debited/credited", "Rs. X debit/credit"
- Extracts **amount** as a float (removes commas from numbers)
- Determines **type** by checking for keywords: "paid", "debited", "debit", "sent" → expense; "received", "credited", "credit" → income
- Extracts **merchant/description** from "at/from/to [Name]" pattern
- Detects payment method: GPay, PhonePe, Paytm — appends to description
- Categorizes based on keywords: food/restaurant/cafe, fuel/petrol/auto, rent/housing, movie/entertainment, shopping/mall, hospital/medical/pharmacy, salary/credited
- Returns `null` if no amount pattern matches

3.9 Validators (schemas.js)

Four Joi schemas that validate request bodies before they reach controllers:

- **userSchema**: email (valid email, required), password (min 6 chars, required), name (required)
- **transactionSchema**: amount (positive number, required), type ('income'|'expense', required), category (required), description (max 500 chars, optional), date (required), walletId (required), currency (default 'INR')
- **bankDetailsSchema**: userId (required), accounts array with objects containing bankName, accountNumber, ifscCode, accountType (all required)

- **smsSchema:** message (required), from (optional), timestamp (optional)



SECTION 4

Frontend – React Application

The React SPA structure, state management, and service layer.

4.1 Main App Component (App.tsx)

`App.tsx` is the root component that acts as the **application controller**. It holds the global state and renders the appropriate page based on the active tab.

State Variables Explained

STATE VARIABLE	TYPE	WHAT IT STORES
<code>user</code>	User null	Currently logged-in user data (id, email, name). null = not logged in
<code>activeTab</code>	string	Currently visible page: 'dashboard', 'wallets', 'transactions', 'insights', 'suggestions', 'investments', 'profile', 'settings'
<code>transactions</code>	Transaction[]	All transactions loaded from the backend or localStorage fallback
<code>wallets</code>	Wallet[]	3 wallets: Main Checking, Cash, Savings. Loaded from localStorage or defaults
<code>budgets</code>	Budget[]	3 budget categories: Food (₹5000), Transport (₹3000), Entertainment (₹2000). Auto-updated based on transactions
<code>isModalOpen</code>	boolean	Controls TransactionModal visibility

STATE VARIABLE	TYPE	WHAT IT STORES
<code>isParserModalOpen</code>	boolean	Controls MessageParserModal (SMS input) visibility
<code>isSMSIntegrationOpen</code>	boolean	Controls SMSIntegrationModal visibility
<code>searchQuery</code>	string	Live search filter text used in Dashboard and Ledger
<code>aiInsights</code>	string	Text response from Gemini for Insights page
<code>aiSuggestions</code>	AISuggestionsResult null	Structured AI suggestions: summary + suggestion cards

useEffect Hooks Explained

1. **Auth Check** (runs once on mount) — Calls `AuthService.getCurrentUser()` to read session from localStorage. If found, sets the user state (auto-login on refresh)
2. **Data Loading** (runs once on mount) — Reads wallets and budgets from localStorage. Fetches transactions from `GET /api/transactions` with JWT. Falls back to localStorage if backend fails. Listens for `refreshTransactions` custom events (fired after bank details save)
3. **Wallet Persistence** — Saves wallets to localStorage whenever they change
4. **Transaction Persistence + Budget Sync** — Saves transactions to localStorage; recalculates each budget's `spent` amount by summing matching expense transactions
5. **Budget Persistence** — Saves updated budgets to localStorage

Key Functions

- `addTransaction(txData)` — optimistically adds to state, POSTs to backend, updates wallet balance
- `deleteTransaction(id)` — removes from state, DELETes from backend
- `handleGenerateInsights()` — calls `getSpendingInsights(transactions)`, stores result
- `handleGenerateSuggestions()` — calls `getAISuggestions(transactions)`, stores structured result
- `handleLogin(user)` / `handleLogout()` — manages auth state and localStorage cleanup

AUTHENTICATION GATE

The very first render check: `if (!user) return <LoginPage onLogin={handleLogin} />` . If no user is in state, the entire app is replaced with the login page. This is a simple but effective client-side route guard.

4.2 TypeScript Types (`types.ts`)

Defines all shared data structures used across the React application:

```

// Transaction type literal
type TransactionType = 'income' | 'expense';

// Expense/income category enum
enum Category {
  FOOD = 'Food', TRANSPORT = 'Transport',
  HOUSING = 'Housing', ENTERTAINMENT = 'Entertainment',
  SHOPPING = 'Shopping', HEALTH = 'Health',
  SALARY = 'Salary', INVESTMENT = 'Investment', OTHER = 'Other'
}

// Wallet (a money container)
interface Wallet {
  id: string;           // '1', '2', '3'
  name: string;         // 'Main Checking', 'Cash', 'Savings'
  type: 'cash' | 'card' | 'savings';
  balance: number;      // Calculated from transactions
  currency: string;     // 'INR'
  color: string;        // Hex color for UI
}

// A single money movement record
interface Transaction {
  id: string;
  amount: number;
  type: TransactionType;
  category: Category | string;
  date: string;         // ISO string
  description: string;
  walletId: string;
  receiptUrl?: string;  // Base64 image of receipt
  receiptMerchant?: string;
  currency: string;
}

// AI receipt analysis result
interface ReceiptAnalysisResult {
  merchant: string; date: string; amount: number;
  category: string; description: string; currency?: string;
}

// Logged-in user
interface User { id: string; email: string; name: string; }

// Budget limit vs actual spending
interface Budget {
  id: string; category: Category | string;
  limit: number; spent: number; currency: string;
}

```

4.3 Client Services

Auth Service (`services/authService.ts`)

A singleton object `AuthService` with these methods:

- `login(email, password)` — POSTs to `/api/auth/login` . On success: saves JWT token to `localStorage['smartspend_token']` and user to `localStorage['smartspend_session_v1']` . Returns the User object
- `register(email, password, name)` — POSTs to `/api/auth/register` . Saves session to localStorage. Returns User
- `logout()` — Removes session and token from localStorage
- `getCurrentUser()` — Reads and parses `localStorage['smartspend_session_v1']`
- `getToken()` — Returns the stored JWT token string
- `updateProfile(userId, name?, password?)` — PUTs to `/api/auth/user/:userId` with JWT auth header. Updates localStorage session with new user data

Gemini Service (`services/geminiService.ts`)

All AI calls go through this service. It initializes the `GoogleGenAI` client with `VITE_GEMINI_API_KEY` .

FUNCTION	AI MODEL	INPUT	OUTPUT
<code>analyzeReceiptImage(base64)</code>	gemini-2.5-flash	Base64 image	ReceiptAnalysisResult (merchant, amount, date, category, description)
<code>getSpendingInsights(transactions)</code>	gemini-2.5-flash	Last 50 transactions (simplified)	Plain text string (max 3 sentences)
<code>getAISuggestions(transactions)</code>	gemini-2.5-flash	Last 60 transactions + financial summary	AISuggestionsResult object with suggestion cards

FUNCTION	AI MODEL	INPUT	OUTPUT
<code>parseBankMessage(message)</code>	gemini-2.0-flash	Raw SMS string	Partial Transaction object (amount, type, description, category)

Structured JSON responses: For `getAISuggestions` and `parseBankMessage`, the service uses Gemini's `responseMimeType: "application/json"` and `responseSchema` to guarantee structured output, avoiding the need to parse free-form text.

AISuggestionsResult structure:

- `summary` : 2-sentence financial health overview
- `totalIncome`, `totalExpenses`, `savingsRate` : calculated client-side before sending to Gemini
- `suggestions[]` : array of `AI Suggestion` objects with: title, description, type ('saving'|'investment'|'warning'), priority ('high'|'medium'|'low'), estimatedImpact (e.g. "Save ₹2,000/month")

SECTION 5

Pages and Components

Detailed explanation of every screen in SpendWiser.

5.1 Login / Register Page (`LoginPage.tsx`)

The **first screen** the user sees. It serves dual purpose — both Login and Registration in a single component, toggling between two modes.

What's on this page:

- **Logo** — Large "S" in an indigo rounded square with a glow effect
- **Title** — Changes between "Neural Entry" (login) and "Create Node" (register)
- **Error Banner** — Red warning box (appears only on errors) with an alert icon
- **Name Field** — Only visible in register mode. Labeled "Subject Name"
- **Email Field** — Labeled "Neural Identity". Has a mail icon inside the input
- **Password Field** — Labeled "Access Protocol". Has a lock icon. Min 6 characters
- **Submit Button** — Shows "Initialize Connection" (login) or "Register Protocol" (register). Shows a spinner while loading
- **Mode Toggle** — Text link to switch between login and register
- **Security Footer** — "Secure Neural Encryption Active" badge at the bottom

How it works:

1. User submits the form → `handleSubmit()` called
2. Based on `isLoginMode`, calls either `AuthService.login()` or `AuthService.register()`
3. On success: calls `onLogin(user)` which sets user state in `App.tsx` → app renders main layout
4. On error: displays the error message in the red banner

5.2 Sidebar Navigation (`Sidebar.tsx`)

The left-side navigation panel with these menu items:

MENU ITEM	TAB VALUE	ICON
Neural Overview	dashboard	BarChart3
Vault Repository	wallets	Landmark
Ledger Feed	transactions	Clock
Gemini Core	insights	Brain
AI Advisor	suggestions	Lightbulb
Investment Plans	investments	TrendingUp
User Profile	profile	User
Settings	settings	Settings

The sidebar is **collapsible on mobile** — a hamburger menu button toggles it. The active item is highlighted with an indigo background and glow effect.

5.3 Dashboard — "Neural Overview" (`Dashboard.tsx`)

The main landing page after login. Displays a real-time financial overview.

Components on this page:

- **Search Bar (sticky)** — Floating glass search input. Filters the transaction list below in real-time by description and category
- **3 Stats Cards:**
 - *Neural Balance* — Net balance (income – expense), shows wallet count
 - *Total Inflow* — Sum of all income transactions, green TrendingUp icon
 - *Total Outflow* — Sum of all expense transactions, red TrendingDown icon
- **Neural Pulse Section** — Budget tracking area. Renders one `LiquidGauge` component per budget. Each gauge shows: category name, amount spent vs limit. Plus an "Add Module" placeholder card.
- **Spending Landscape** — Recharts `BarChart` with one bar per spending category. Each bar gets a unique color from a 7-color palette. Custom styled tooltip.

- **Live Feed** — A scrollable mini transaction list sorted newest-first. Each item shows: icon (up/down), description, date, amount. At the bottom: "Export Ledger" button → exports CSV

LiquidGauge Component (`LiquidGauge.tsx`)

A custom animated circular gauge that visually fills up based on spending percentage:

- Renders as an SVG with a circular arc path
- Fill color changes: green (under 70%), amber (70-90%), red (above 90%)
- Shows: category label, amount spent, limit

5.4 Vault Repository — "Wallets" (`VaultsView.tsx`)

Displays all wallet cards in a 3-column grid.

Each wallet card shows:

- Type icon (CreditCard/Wallet/Landmark) in the top-left
- Live status badge ("Linked" with a pulsing green dot)
- Wallet type label ("card node", "cash node", etc.)
- Wallet name as the large heading
- "Bitstream Balance" label with the current calculated balance
- Activity indicator at the bottom

Cards have a **holographic depth effect**: a blurred colored blob in the background, a watermark icon, hover animations (scale + rotation), and glassmorphism panel styling.

At the end of the grid, there's an "Initialize New Node" placeholder card that could be used to add wallets in future development.

5.5 Ledger Feed — "Transactions" (`LedgerView.tsx`)

A full-featured transaction list view with search, export, and delete functionality.

Header section:

- Title "Ledger Feed" with a clock icon
- Search input: "Query Ledger Bitstream..."
- **Sort Protocol** button (UI only, no sort logic implemented)

- **Export CSV** button — generates a .csv in the browser and triggers download
- **Download Excel** link — fetches `/api/transactions/download` and downloads .xlsx

Transaction List:

- Each row (transaction) shows: type icon (emerald up / rose down), description (bold), date badge, category badge (indigo pill), amount, and a trash icon (appears on hover)
- Hover effect: row background highlights, left border turns indigo, description glows
- Trash icon calls `onDelete(tx.id)` which propagates to App.tsx's `deleteTransaction`
- Empty state shows a centered empty message

Stats Footer:

4 mini-cards: Total Entries count, "100%" Success Rate (static), "AES-256" Encryption label, "FIX.5 / JSON" Protocol label (branding/aesthetic)

5.6 Gemini Core — "AI Insights" (`InsightsView.tsx`)

A page dedicated to single-paragraph AI-generated spending insights.

Layout:

- **Hero Section** — Large dark gradient panel with animated glowing background, a Brain icon, title "Gemini Neural Core", and the "Ignite AI Intelligence" button
- **Insights Panel** (left 2/3) — After clicking, displays the Gemini response in a large text box with a fingerprint icon header. Shows confidence score (98.4%) and compute latency (420ms) — these are static UI values for aesthetics
- **Info Sidebar** (right 1/3) — Two info cards: "Pattern Recognition" and "Threat Mitigation" (explains AI capabilities). Plus a "Pro Linked" subscription badge

Button behavior:

- Disabled if user has no transactions (shows nothing to analyze)
- Shows a spinning loader while Gemini is processing
- Displays the plain text response once received

5.7 AI Advisor — "Suggestions" (`AISuggestionsView.tsx`)

More detailed and structured AI financial advice than the Insights page.

Layout after generating:

- **Hero** — Emerald-themed panel with a Lightbulb icon and "Get AI Suggestions" button
- **3 Stats Row:** Total Income, Total Expenses, Savings Rate% (calculated client-side)
- **Financial Health Summary** — The 2-sentence summary from Gemini
- **Suggestions Grid** — 3 sections: Warnings (rose), Saving Tips (emerald), Investment Opportunities (indigo). Each section shows 2-column card grid

SuggestionCard Component:

- Each card shows: type icon, priority badge (high=rose, medium=amber, low=gray), type label
- Title in bold uppercase
- 2-3 sentence description
- "Estimated Impact" pill at the bottom (e.g., "Save ₹2,000/month")
- Cards scale up slightly on hover

5.8 Investment Plans (`InvestmentPlansView.tsx`)

Displays 6 curated Indian investment options with real data, personalized recommendations based on the user's savings rate.

On load:

- Fetches `GET /api/investment-rates/scrape` to get live/cached rates
- Calculates monthlyIncome, monthlyExpenses, monthlySavings, savingsRate from current month's transactions

Recommendation Algorithm:

SAVINGS RATE	RECOMMENDED PLANS
≥ 30%	Mutual Funds + NPS
20–29%	PPF + NPS
10–19%	PPF + Fixed Deposit
< 10%	Fixed Deposit + Gold

SAVINGS RATE	RECOMMENDED PLANS
Monthly savings $\geq$ ₹10,000	Also adds LIC

6 Investment Plans:

PLAN	MIN INVESTMENT	RISK	TIME HORIZON	TAX BENEFIT
PPF	₹500	Low	15 years	₹1.5L under 80C
Mutual Funds (Equity)	₹100	High	5-10 years	LTCCG 20% after 1 yr
LIC Insurance	₹5,000	Low	10-20 years	₹1.5L under 80C
NPS	₹500	Medium	20-30 years	80C + 80CCD(1B)
Fixed Deposits	₹1,000	Low	1-10 years	Taxed as income
Gold Investment	₹100	Medium	5-10 years	LTCCG 20% after 2 yrs

Each plan card shows: icon, recommended badge (if applicable), name, description, min investment, expected return (live rate if available), time horizon, risk level badge, key benefits list, tax benefit, and an "Explore" / "Learn More" button.

A **disclaimer box** at the bottom clarifies these are informational only.

5.9 User Profile (`UserProfileView.tsx`)

Displays and edits personal and bank information.

Profile Header:

- Large avatar showing the first letter of the user's name (e.g., "A" for Anjali)
- Name (large, uppercase), email, "Account Active" green status badge

Personal Information Section:

- Read-only display of Full Name, Email, Account ID (UUID)

- Edit icon that was present for future implementation (no edit logic currently active in this section)

Bank Details Section (up to 10 accounts):

- **View mode:** Shows each bank account card with: Account Holder, Bank Name, Account Number (masked: ****1234), IFSC Code, Account Type, Mobile Number
- **Edit mode** (click pencil icon): Converts to form inputs for each account. "+" button adds more accounts. Trash icon removes individual accounts. "Save All Details" button POSTs to `/api/bank-details`, fires `refreshTransactions` event to trigger demo data loading
- Supports adding up to 10 bank accounts simultaneously
- Account numbers entered as password fields for privacy

Security Note:

An amber-colored callout box explains that bank details are stored locally and never sent to external servers (they are actually sent to the backend, but this note is for user reassurance).

5.10 Settings (`SettingsView.tsx`)

App configuration page with 4 sections, all persisted to `localStorage['app_settings']`.

Notifications & Alerts:

- **Push Notifications** toggle — On enable, requests OS permission via `Notification.requestPermission()`
- **Email Alerts** toggle — Weekly spending summaries (notification only, not yet implemented)
- **Sound Effects** toggle — On enable, plays a test beep using the Web Audio API (880Hz sine wave for 100ms)

Display & Theme:

- **Dark Mode** — Always ON (static, cannot be disabled — the entire app is dark-mode only)
- **Currency** dropdown — INR, USD, EUR, GBP (stored in settings; visual preference only)

Security & Privacy:

- **Two-Factor Authentication** toggle (stored preference; not yet implemented)

- **Auto-Sync** toggle (stored preference)
- **Data Retention** dropdown — 30 days, 90 days, 6 months, 1 year, 2 years (stored preference)

Data Management:

- **Export Data** — Fetches transactions from backend (or localStorage fallback), combines with wallets, budgets, settings, and downloads as `spendwiser-backup-YYYY-MM-DD.json`
- **Clear All Data** — Confirms with user, calls `DELETE /api/data/clear`, removes all localStorage keys, reloads page

A green "Settings Saved" toast notification appears at the bottom-right for 2 seconds whenever settings change.

5.11 Transaction Modal (`TransactionModal.tsx`)

A full-screen modal overlay that allows users to log a new transaction.

Fields:

- **Type Toggle** — Two buttons: "System Drain" (expense, rose) / "System Inflow" (income, emerald)
- **Amount** — Number input with ₹ prefix, 2 decimal places, required
- **Target Node** (Wallet) — Dropdown with wallet names
- **Description** — Text input, required
- **Classification** (Category) — Dropdown with all Category enum values
- **Timestamp** (Date) — Date picker, defaults to today
- **Neural Scan** (Optional Receipt) — Camera button opens file picker for image; Gemini AI auto-fills amount, category, description, date from the image

Buttons:

- "Abort Entry" → closes modal
- "Authorize Protocol" → validates and submits

The modal appears with a backdrop blur effect and animates in. A light scan animation runs across the submit button on hover.

5.12 SMS Parser Modal (`MessageParserModal.tsx`)

Allows users to paste a bank SMS text and automatically create a transaction from it.

Process:

1. User clicks "Decrypt SMS" in the header
2. Modal opens with a textarea for SMS input
3. User pastes SMS text (e.g., "Rs. 500 debited from your HDFC account...")
4. Clicks "Parse SMS" → calls `parseBankMessage(text)` from `geminiService` (uses Gemini 2.0 Flash)
5. Gemini returns parsed data → pre-fills the transaction form fields
6. User reviews and confirms → transaction is saved

SECTION 6

API Reference

Every endpoint, its request format, and response structure.

All API endpoints are prefixed with `/api` and the server runs on `http://localhost:5000` by default in development.

AUTHENTICATION

Protected endpoints require: `Authorization: Bearer <jwt-token>` in the request headers. The JWT token is obtained from the login response.

6.1 Authentication Endpoints

POST `/api/auth/register`

Rate limit: 3 registrations per hour per IP

Request body:

```
{ "email": "user@example.com", "password": "min6chars", "name": "Alice" }
```

Success (201):

```
{ "id": "uuid-string", "email": "user@example.com", "name": "Alice" }
```

Error (400): `{ "error": "User already exists" }`

Error (503): `{ "error": "MongoDB not connected. ..." }`

POST `/api/auth/login`

Rate limit: 5 attempts per 15 minutes (skips counting successful logins)

Request body:

```
{ "email": "user@example.com", "password": "yourpassword" }
```

Success (200):

```
{ "token": "eyJhbGciOiJIUzI1NiIs... ", "user": { "id": "uuid", "email": "...", "
```

Error (401): `{ "error": "Invalid email or password" }`

GET /api/auth/user/:userId

Auth: Required | Returns user object for given userId

PUT /api/auth/user/:userId

Auth: Required | **Body:** `{ "name": "New Name", "password": "newpass" }` (both optional)

6.2 Transaction Endpoints

GET /api/transactions

Auth: Required | Returns all transactions sorted by date descending.

```
[{ "id": "uuid", "amount": 500, "type": "expense", "category": "Food", "descrip
```

POST /api/transactions

Auth: Required | **Body:**

```
{ "amount": 500, "type": "expense", "category": "Food", "description": "Swiggy" }
```

Success (201): Returns the created transaction with auto-generated `id`

PUT `/api/transactions/:id`

Auth: Required | Send partial or full transaction fields to update

DELETE `/api/transactions/:id`

Auth: Required | Returns `{ "message": "Transaction deleted" }`

GET `/api/transactions/download`

Auth: Not required | Downloads `spendwiser-transactions.xlsx` file

DELETE `/api/transactions/data/clear`

Auth: Not required | Deletes all transactions from MongoDB AND clears Excel file. Use with extreme caution!

6.3 Bank Details Endpoints

GET `/api/bank-details/:userId`

Auth: Required | Returns the BankDetails document for the userId

POST `/api/bank-details`

Auth: Required | Upsert operation (creates or replaces)


```
{ "userId": "uuid", "accounts": [{ "bankName": "HDFC", "accountNumber": "123456
```

On first save with accounts present, creates 4 sample transactions tagged `source: "bank_sync"`

DELETE /api/bank-details/:userId

Auth: Required | Removes bank details for the user

6.4 SMS Webhook

POST /api/sms/webhook

Auth: Not required | **Rate limit:** 50 per hour

```
{ "message": "Rs.500 debited from your account. Balance: Rs.25000", "from": "+9
```

Success (201):

```
{ "success": true, "transaction": { "id": "...", "amount": 500, "type": "expens
```

Error (400): `{ "error": "Could not parse transaction from SMS" }` (if no regex pattern matched)

6.5 Investment Rates

GET /api/investment-rates/scrape

Auth: Not required | Returns cached investment rates instantly (no scraping on request)

```
{ "ppf": "7.1% p.a. (Live)", "fixed-deposit": "6.5% p.a.", "nps": "9.0% p.a.",
```

SECTION 7

Security

How SpendWiser protects user data at every layer.

SECURITY LAYER	MECHANISM	DETAILS
Password Storage	bcryptjs hashing	Passwords are hashed with bcrypt (salt rounds = 10) before storage. Plain passwords NEVER saved.
Authentication	JWT Bearer Token	Login returns a signed JWT with 7-day expiry. All protected routes verify this token via middleware.
Rate Limiting	express-rate-limit	General: 100 req/15min. Login: 5 attempts/15min. Register: 3/hour. SMS: 50/hour.
CORS Policy	cors package	Only allows requests from the configured FRONTEND_URL. Credentials enabled for cookie support.
Input Validation	Joi schemas	All API inputs are validated before reaching business logic — prevents malformed data injection.
Account Masking	Client-side	Bank account numbers shown as ****XXXX (last 4 digits only) in the UI.
Local Session	localStorage	Session stored client-side only. No server-side session storage reduces attack surface.
Error Handling	Global middleware	All unhandled errors return generic "Internal Server Error" — no stack traces exposed to clients.

⚠️ KNOWN SECURITY NOTES FOR PRODUCTION

- The `JWT_SECRET` defaults to 'your-secret-key' if not set. **Always set a strong secret in .env for production.**

- The `/api/transactions/download` and `/api/transactions/data/clear` endpoints do not require authentication — they should be protected in a production environment.
- The Gemini API key is exposed client-side via `VITE_GEMINI_API_KEY`. For production, consider proxying AI calls through the backend to keep the key server-side.

SECTION 8

Data Storage & Backup

Where every piece of data lives and how it's protected.

DATA TYPE	PRIMARY STORAGE	FALLBACK	SECURITY LEVEL
User accounts (email, hashed password, name)	MongoDB — users collection	None (requires DB)	★★★★★
Transactions (amount, type, category, date)	MongoDB — transactions collection	data.xlsx Excel file	★★★★★
Bank details (account number, IFSC, etc.)	MongoDB — bankdetails collection	localStorage (in edit mode)	★★★★★
Investment rates (PPF, FD, etc.)	MongoDB — rates collection	rates.json file	★★★
Wallets (name, balance, color)	Browser localStorage	Default hardcoded values	★★★
Budgets (category, limit, spent)	Browser localStorage	Default hardcoded values	★★★
Settings (notifications, currency, etc.)	Browser localStorage	Default hardcoded values	★★★
Auth session (user object)	Browser localStorage	N/A	★★★

DATA TYPE	PRIMARY STORAGE	FALLBACK	SECURITY LEVEL
JWT token	Browser localStorage	N/A	☆☆☆

localStorage Keys Used by SpendWiser

KEY	CONTENTS
<code>smartspend_session_v1</code>	JSON-stringified User object (id, email, name)
<code>smartspend_token</code>	JWT Bearer token string
<code>app_transactions_v2</code>	Array of Transaction objects (localStorage cache, synced from DB)
<code>app_wallets_v2</code>	Array of Wallet objects
<code>app_budgets_v2</code>	Array of Budget objects with live spent amounts
<code>app_settings</code>	AppSettings object (notifications, theme, etc.)
<code>app_bank_details_list</code>	Array of BankDetails objects (local copy)

Excel Backup (data.xlsx)

The file `backend/data.xlsx` serves as a **persistent offline backup** for transactions when MongoDB is unavailable. It has one sheet "Transactions" with columns: id, amount, type, category, description, date, walletId, currency, createdAt, source, smsFrom.

It is also available for direct download via `GET /api/transactions/download`.

✓ DATA EXPORT

Users can export a full JSON backup of all their data from Settings → Export Data. The file contains: transactions (from DB or localStorage), wallets, budgets, settings, and export date.

File name: `spendwiser-backup-YYYY-MM-DD.json`

SECTION 9

Deployment

How to host SpendWiser on Vercel for free.

Vercel Deployment Architecture

SpendWiser is configured for **zero-configuration Vercel deployment** using `vercel.json`:

```
{
  "outputDirectory": "client/dist",    // built React app
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/index" }, // backend API
    { "source": "/(.*)", "destination": "/index.html" }     // SPA routing
  ]
}
```

How it works on Vercel:

1. `client/dist` — the built React app (Vite output) is served as static files
2. All `/api/*` requests are routed to `api/index.js` which re-exports the Express app as a Vercel Serverless Function
3. All other routes redirect to `index.html` — enabling client-side React Router navigation

Deployment Steps:

1. Push the repository to GitHub
2. Import project into Vercel Dashboard (vercel.com)
3. Add Environment Variables in Vercel Settings:
 - `MONGODB_URI` — MongoDB Atlas connection string
 - `VITE_GEMINI_API_KEY` — Google Gemini API key
 - `JWT_SECRET` — Strong random string (e.g., 64 chars)
4. Deploy — Vercel auto-builds and deploys

⚠️ IMPORTANT FOR VERCEL DEPLOYMENT

- On Vercel, the backend runs as a **serverless function** — there is no persistent server process. The Excel file fallback (`data.xlsx`) uses the `/tmp` directory which is ephemeral (wiped between invocations)
- You **must** use MongoDB Atlas (cloud) on Vercel — local MongoDB won't work
- The scraping cron job won't run on Vercel serverless — you'd need a separate cron service or Vercel Cron features

Local Development Architecture

In development, the frontend (Vite, port 3000/3001) and backend (Express, port 5000) run as separate processes. Vite's dev server proxies `/api` requests to Express. The root `package.json` scripts use `concurrently` to run both simultaneously.

```
npm run dev    # runs both frontend and backend concurrently
```


SECTION 10

Setup & Installation Guide

Step-by-step guide to run SpendWiser locally.

Prerequisites

- **Node.js** v18 or higher — nodejs.org
- **MongoDB** (local) — Community Edition, OR MongoDB Atlas (cloud) — mongodb.com
- **Google Gemini API Key** (free) — aistudio.google.com
- A modern web browser (Chrome, Firefox, Edge)

Step 1: Clone/Download the Project

```
git clone https://github.com/Varshitha61/Spend-Wiser.git
cd spendwiser
```

Step 2: Install All Dependencies

```
# Install root + frontend dependencies
npm install

# Install backend dependencies
cd backend
npm install
cd ..
```

Step 3: Configure Backend Environment

Create/edit `backend/.env` :

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/spendwiser
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production
JWT_EXPIRE=7d

# Optional: Email (for welcome emails)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-gmail-app-password

# Frontend URL (for CORS)
FRONTEND_URL=http://localhost:3001
```

Step 4: Configure Frontend Environment

Create/edit `client/.env.local` :

```
VITE_GEMINI_API_KEY=your-gemini-api-key-here
VITE_API_BASE_URL=http://localhost:5000
```

Step 5: Start MongoDB

```
# Windows (MongoDB should auto-start as a service)
# If not: net start MongoDB

# Mac
brew services start mongodb-community

# Linux
sudo systemctl start mongod

# Or use MongoDB Compass GUI to connect
```

Step 6: Start the Application

```
# From the root directory – starts both backend (port 5000) and frontend (port 3000/3001)
npm run dev
```

Step 7: Open in Browser

- **Frontend:** `http://localhost:3001` (or 3000)
- **Backend API:** `http://localhost:5000`
- Register a new account or use a test account

✓ QUICK VERIFICATION

Open your browser to the frontend URL. You should see the SpendWiser login page with a dark background. Try registering with a test email. If the backend is running correctly, you'll be logged in and see the dashboard. The console should show: 🚀 `Server running on port 5000` and ✓ `MongoDB connected successfully`

SECTION 11

Environment Variables

Complete reference of all configuration variables.

Backend (.env)

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
PORT	No	5000	Port the Express server listens on
MONGODB_URI	Yes	mongodb://localhost:27017/spendwiser	MongoDB connection string. Use Atlas URI for cloud
JWT_SECRET	Yes	'your-secret-key' (unsafe!)	Secret key for signing/verifying JWT tokens. Use 64+ random chars in production
JWT_EXPIRE	No	'7d'	JWT token expiry duration
FRONTEND_URL	No	http://localhost:3001	Allowed origin for CORS requests
SMTP_HOST	No	smtp.gmail.com	SMTP server for sending welcome emails
SMTP_PORT	No	587	SMTP port (587=TLS,

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
			465=SSL)
SMTP_USER	No	—	SMTP auth username (email address). If not set, email sending is skipped
SMTP_PASS	No	—	SMTP auth password (use Gmail App Password, not regular password)
NODE_ENV	No	'development'	Set to 'production' on Vercel to prevent server.listen() (Vercel manages this)
VERCEL	Auto	—	Set automatically by Vercel to '1'. Used to switch Excel storage to /tmp dir

Frontend (client/.env.local)

VARIABLE	REQUIRED	DESCRIPTION
VITE_GEMINI_API_KEY	Yes (for AI)	Google Gemini API key. Get free key from Google AI Studio. Without this, all AI features are disabled.
VITE_API_BASE_URL	No	Backend API base URL for development

● SECURITY WARNING

- Never commit `.env` files to Git (they are already in `.gitignore`)
- The `VITE_GEMINI_API_KEY` is included in the browser bundle — anyone can see it using DevTools. Restrict API key usage in Google Cloud Console to prevent abuse.
- Always use a strong `JWT_SECRET` in production — at least 64 random characters

SECTION 12

Troubleshooting

Common issues and how to fix them.

PROBLEM	SYMPTOMS	SOLUTION
MongoDB Connection Error	"MongoDB not connected" in API responses; ECONNREFUSED error in console	1. Ensure MongoDB service is running (<code>mongosh</code> to test) 2. On Windows: Services → MongoDB → Start 3. If using Atlas: Check connection string, whitelist your IP 4. On some ISPs: DNS blocks MongoDB SRV. Try Google DNS (8.8.8.8) 5. App will fallback to Excel storage automatically
Port Already in Use	"EADDRINUSE: address already in use :::5000"	1. Kill the process using port 5000: <code>netstat -ano findstr :5000</code> then <code>taskkill /PID [pid] /F</code> 2. Or change PORT in backend/.env
AI Features Not Working	No insights generated, buttons do nothing	1. Check that <code>VITE_GEMINI_API_KEY</code> is set in <code>client/.env.local</code> 2. Check browser console for "VITE_GEMINI_API_KEY is not set" warning 3. Verify the key is valid at aistudio.google.com
Login Returns 401	"Invalid email or password" even with correct credentials	1. Ensure MongoDB is connected (user data lives in DB) 2. Check if you registered with the exact same email (case-insensitive) 3. Try registering a new account
CORS Error	"Access-Control-Allow-Origin" error in	1. Set <code>FRONTEND_URL</code> in backend/.env to match your frontend URL exactly

PROBLEM	SYMPTOMS	SOLUTION
	browser console	2. Ensure backend is running before frontend 3. Check Vite proxy config in <code>vite.config.ts</code>
SMS Parsing Fails	"Could not parse transaction from SMS"	1. The SMS format must contain "Rs.", "₹", or "INR" followed by a number 2. Must contain "debited"/"paid"/"received"/"credited" keywords 3. Try using the AI parser (Gemini) instead — it handles more formats
Transactions Not Loading	Dashboard shows empty, no transactions visible	1. Check backend is running on port 5000 2. Open browser DevTools → Network → check <code>/api/transactions</code> response 3. If 401: ensure you're logged in and JWT token is valid 4. Check localStorage for <code>'app_transactions_v2'</code> as cache
Missing Dependencies	"Cannot find module 'express'" or similar errors	Run: <code>npm install</code> in both root and <code>backend/</code> directories
Vercel Build Fails	Build error on Vercel dashboard	1. Ensure all environment variables are set in Vercel project settings 2. Check that <code>client/dist</code> is in <code>outputDirectory</code> in <code>vercel.json</code> 3. Check build logs for specific TypeScript errors

Debug Tips

1. Open browser Developer Tools (F12) → Console tab — any errors will appear here
2. Check the Network tab → look at API calls to `/api/*` — check status codes and response bodies
3. Check the backend terminal for server-side errors
4. Test API endpoints directly with a tool like Postman or curl


```
# Test backend is running
curl http://localhost:5000/api/investment-rates/scrape

# Test login (should return token)
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"password123"}'
```

CONGRATULATIONS!

You've read the complete SpendWiser technical documentation. You now understand every file, every endpoint, every component, and every concept in this project. Feel free to extend the app — add new features, fix bugs, or use it as a learning resource for MERN stack development with AI integration.

SpendWiser — Personal Finance Management App

 Technical Documentation v2.5.0 | June 2026

Built with  using React, Node.js, Express, MongoDB, and Google Gemini AI